

Taller Práctico — Unidad 3

Introducción a Bases de Datos NoSQL (MongoDB)

Configuración del entorno — Docker

Se levantó el entorno de MongoDB utilizando Docker Compose. El contenedor **mongodb_taller** fue iniciado correctamente junto con la red y el volumen persistente.

```
● aj@dynamic-ip-adsl docker-mongo % docker-compose up -d
[+] up 14/14
  ✓ Image mongo:latest           Pulled           563.6s
  ✓ Network docker-mongo_default Created         0.0s
  ✓ Volume docker-mongo_mongodb_data Created         0.0s
  ✓ Container mongodb_taller     Started         0.4s
○ aj@dynamic-ip-adsl docker-mongo %
```

Figura: Ejecución de `docker-compose up -d` — contenedor `mongodb_taller` iniciado correctamente.

Actividad 1 — Diseño del documento

1. Información para embebir

Se considera adecuado embebir la información técnica del equipo (marca, modelo, potencia) y el **historial de mantenimientos**. Esto permite que cada vez que se consulte un equipo, se obtenga su trayectoria completa de mantenimiento sin necesidad de consultar otras colecciones.

2. Información para separar

Se podría separar la información detallada de los **técnicos** (datos personales, certificaciones) y el **catálogo de repuestos**. Esto evitaría redundancia si muchos equipos comparten los mismos técnicos o repuestos.

Justificación

El modelo de documentos es ideal porque permite agrupar datos que se leen juntos frecuentemente. Embeber los mantenimientos optimiza el rendimiento al evitar *joins* costosos, y la flexibilidad de MongoDB permite que cada registro varíe en estructura según sea necesario, adaptándose perfectamente a la realidad industrial.

Actividad 2 — Ejemplo de documento

A continuación se presenta el documento JSON insertado en la colección `equipos` de la base de datos `taller_mantenimineto`:

```
{
  "nombre": "Generador Eléctrico G-502",
  "tipo": "Generador Diesel",
  "ubicacion": "Planta Industrial Sur",
  "estado": "Operativo",
  "mantenimientos": [
    {
      "fecha": "2026-03-15",
      "tecnico": "Carlos Rodríguez",
      "observacion": "Cambio de aceite y filtros."
    },
    {
      "fecha": "2026-04-20",
      "tecnico": "Andrea Méndez",
      "observacion": "Revisión de alternador."
    }
  ]
}
```

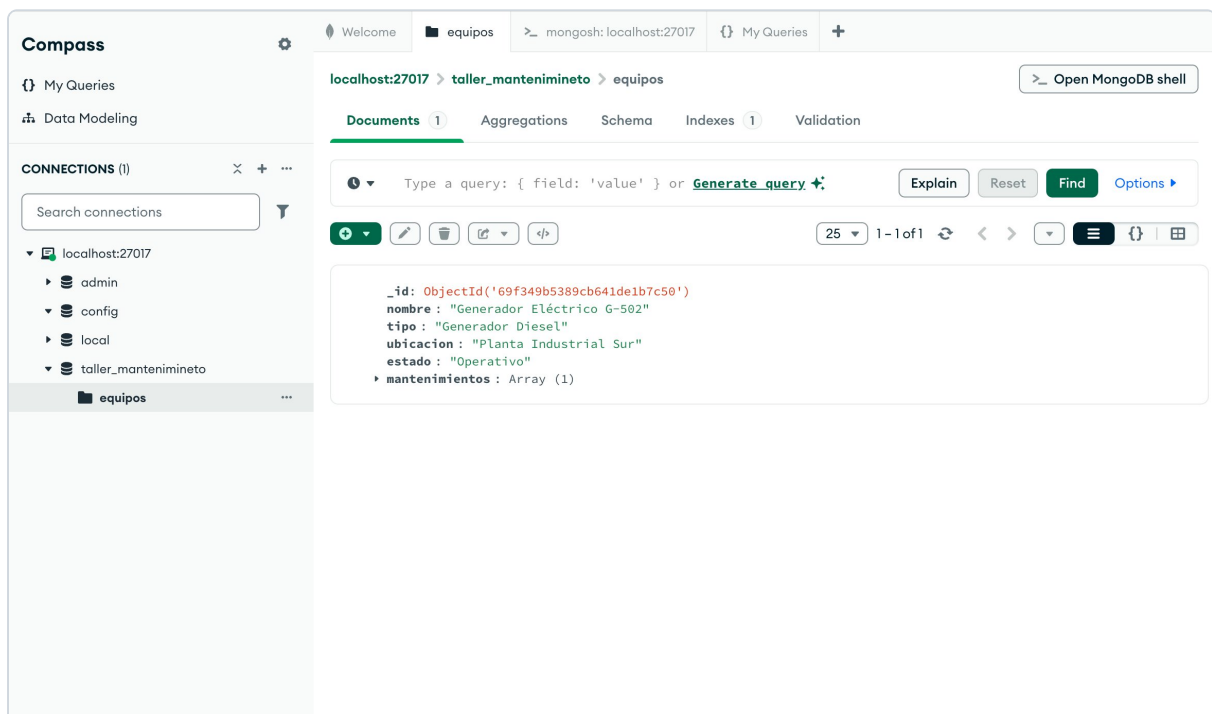


Figura: Documento insertado visible en MongoDB Compass — colección equipos, 1 documento.

Actividad 3 — Consultas básicas

1. Obtener todos los equipos de una sede específica:

```
db.equipos.find({ "ubicacion": "Planta Industrial Sur" })
```

2. Buscar equipos por nombre (palabra clave):

```
db.equipos.find({ "nombre": { "$regex": "Generador", "$options": "i" } })
```

3. Equipos con mantenimiento de un técnico específico:

```
db.equipos.find({ "mantenimientos.tecnico": "Andrea Méndez" })
```

Actividad 4 — Actualización simple

1. Cambiar el estado de un equipo:

```
db.equipos.updateOne(  
  { "nombre": "Generador Eléctrico G-502" },  
  { "$set": { "estado": "En Reparación" } }  
)
```

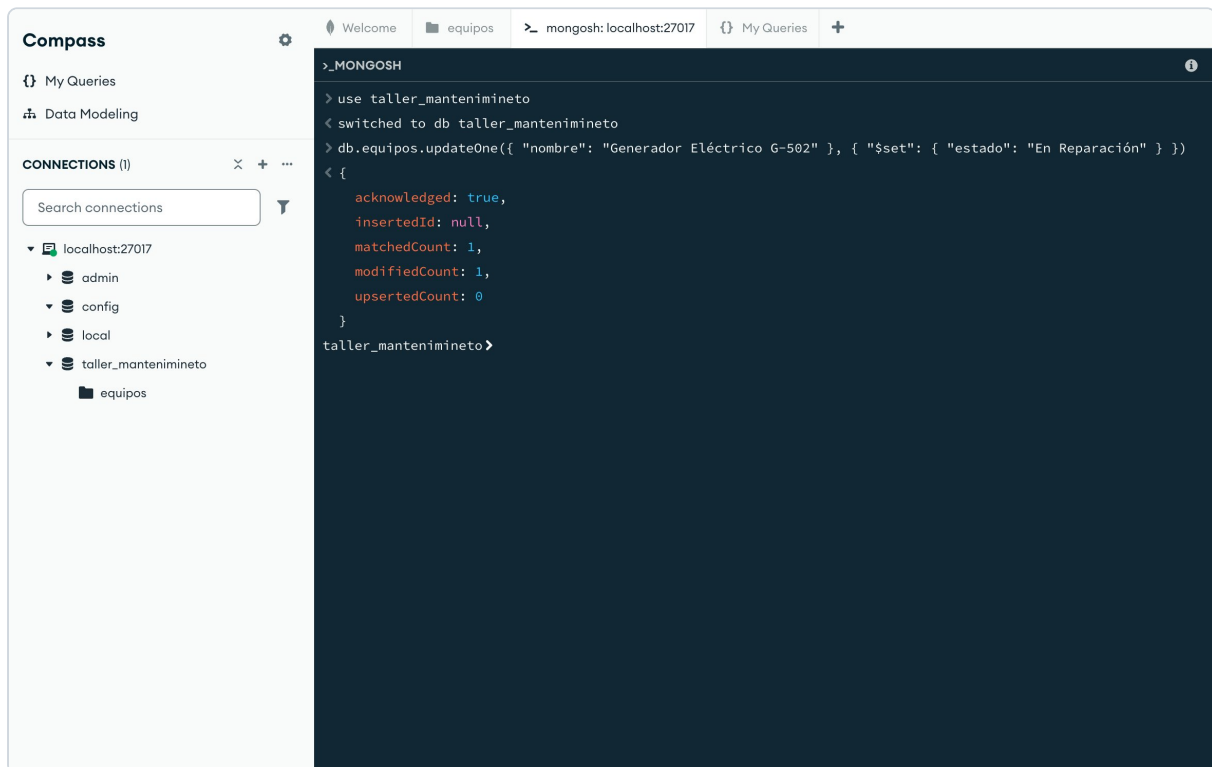


Figura: Ejecución del updateOne — matchedCount: 1, modifiedCount: 1. Estado actualizado a "En Reparación".

2. Agregar un nuevo registro de mantenimiento:

```
db.equipos.updateOne(
  { "nombre": "Generador Eléctrico G-502" },
  {
    "$push": {
      "mantenimientos": {
        "fecha": "2026-04-30",
        "tecnico": "Ricardo Soto",
        "observacion": "Revisión preventiva de bornes de batería."
      }
    }
  }
)
```

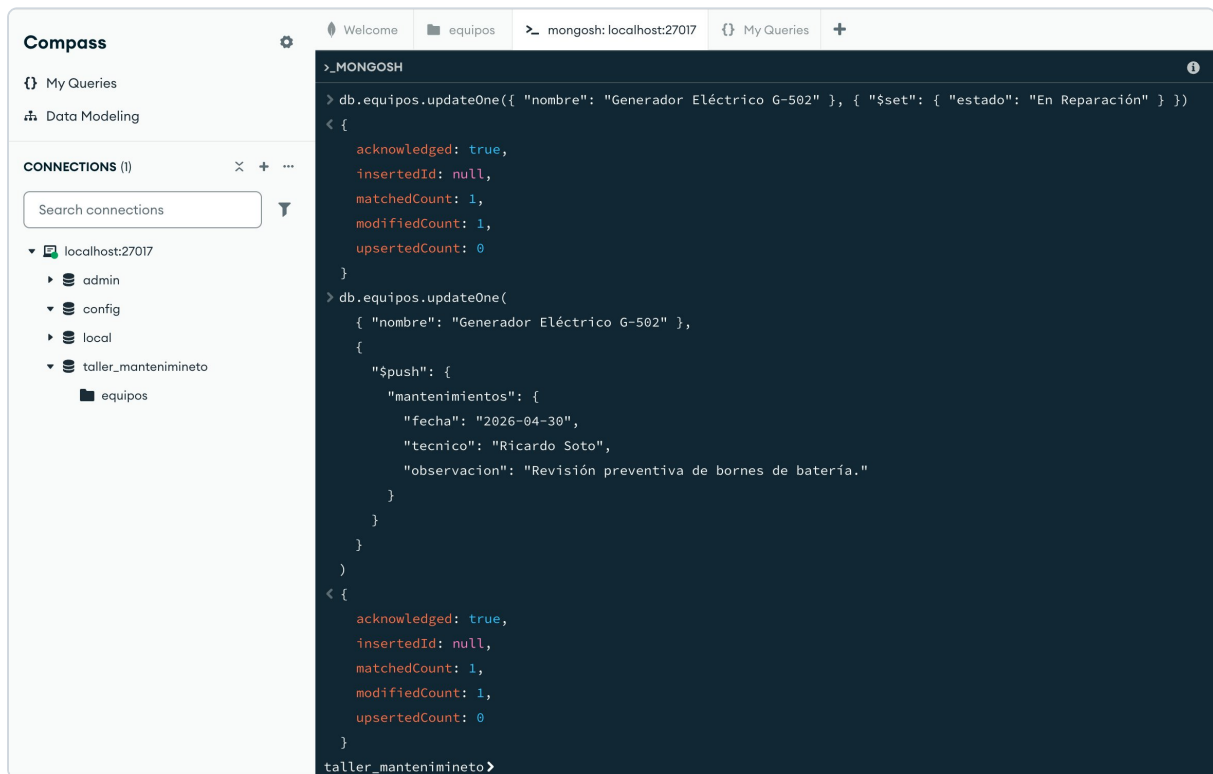


Figura: Ejecución del \$push — nuevo mantenimiento agregado correctamente (modifiedCount: 1).

Resultado final en Compass:

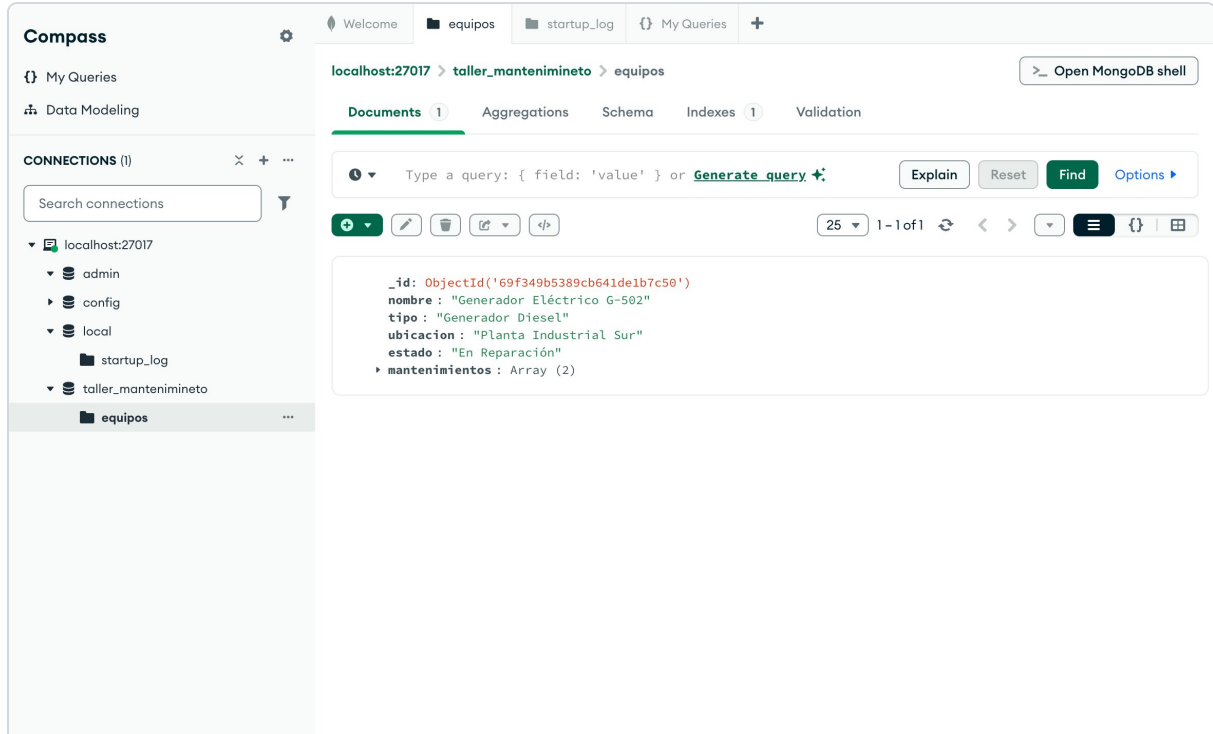


Figura: Estado final del documento — estado "En Reparación" y mantenimientos: Array (2).

Actividad 5 — Reflexión breve

¿Por qué el modelo de documentos es adecuado?

Es adecuado porque la información de mantenimiento es jerárquica por naturaleza (un equipo tiene muchos mantenimientos) y su estructura puede cambiar. MongoDB permite representar esta relación de forma directa y eficiente.

¿Qué ventaja ofrece frente a un modelo relacional?

La principal ventaja es la **flexibilidad del esquema** y la **velocidad de consulta**. En un sistema relacional, se requerirían múltiples tablas y uniones para obtener el mismo resultado, lo que complica el desarrollo y reduce el rendimiento a medida que los datos crecen.